

課程作業：CSCS20024 HW1：Dataset Splitting & Avoiding Data Leakage

學號：113511103

姓名：吳沛恩

系級：醫學系二年級

繳交日期: 2025 年 9 月 9 日

一、作業內容簡介

本次作業的目標是學習正確的資料集分割程序，並理解 Data Leakage 的風險。為此我們實作了一個自定義的資料載入器。其中 `dataset_loader.py` 用以處理 MNIST 資料集。

實作方法與流程如下：

1. 資料讀取

- 首先讀取 MNIST 資料集，此資料集被整理在 10 個獨立的資料夾中，每個資料夾對應一個數字標籤 (0-9)。
- 程式會遍歷這些資料夾，讀取圖片檔案的路徑，並根據其所在資料夾賦予正確標籤。

```
1 import os
2 import random
3
4 def load_and_split_dataset(base_dir, train_ratio=0.7, val_ratio=0.15):
5
6     all_files_by_label = {}
7
8     # 1. Loading all files from data / sort by labels
9     print(f"Loading data from: {os.path.abspath(base_dir)}")
10    if not os.path.exists(base_dir):
11        print(f"Error: Dataset directory not found at '{os.path.abspath(base_dir)}'")
12        return [], [], []
13
14    for label_folder in sorted(os.listdir(base_dir)):
15        label_path = os.path.join(base_dir, label_folder)
16        if os.path.isdir(label_path):
17            try:
18                digit = int(label_folder.split('_')[1])
19                images = [os.path.join(label_folder, f) for f in sorted(os.listdir(label_path))]
20                all_files_by_label[digit] = images
21            except (ValueError, IndexError):
22                continue
23
24    train_list = []
25    val_list = []
26    test_list = []
```

▲ 圖片一：資料載入與讀取

2. 自定義分割邏輯

- 為遵循作業要求，我們並未使用任何內建的隨機分割函數。
- 為了避免類別不平衡，此 70/15/15 的分割比例被嚴格應用於每一個類別。

```

28     # 2. Categorize images by digits label
29     for digit, images in all_files_by_label.items():
30         random.shuffle(images)
31
32         n_total = len(images)
33         n_train = int(n_total * train_ratio)
34         n_val = int(n_total * val_ratio)
35
36         class_train = images[:n_train]
37         class_val = images[n_train : n_train + n_val]
38         class_test = images[n_train + n_val :]
39
40         train_list.extend([(img_path, digit) for img_path in class_train])
41         val_list.extend([(img_path, digit) for img_path in class_val])
42         test_list.extend([(img_path, digit) for img_path in class_test])

```

▲ 圖片二：資料分割處理

3. 避免資料洩漏

- 為防止資料洩漏，分割步驟在資料載入後立即執行。
- 在分割完成前，沒有進行任何如 Normalization 等預處理步驟。
- 此方法確保了訓練、驗證與測試集三者之間完全獨立，沒有任何重複的圖片。

4. 輸出格式

- 程式執行後，會將分割結果儲存至三個獨立的文字檔案中
- 最後，程式會在終端機印出每個集合的樣本總數。

```

44     # 3. Write in file
45     def save_split(filepath, data_list):
46         with open(filepath, 'w') as f:
47             for img_path, label in data_list:
48                 f.write(f"{img_path.replace(os.sep, '/')} {label}\n")
49
50     save_split('train_list.txt', train_list)
51     save_split('val_list.txt', val_list)
52     save_split('test_list.txt', test_list)
53
54     # 4. Output results in terminal
55     print("\nDataset splitting complete.")
56     print(f"Train set size: {len(train_list)}")
57     print(f"Validation set size: {len(val_list)}")
58     print(f"Test set size: {len(test_list)}")
59
60     return train_list, val_list, test_list

```

▲ 圖片三：以正確格式輸出並在終端機顯示結果

5. 洩漏驗證

- 作為 Bonus 我們額外實作了 check_data_leakage() 函數，用以程式化地驗證三個集合中沒有任何樣本重疊，再次確認了資料的獨立性。

```

62 def check_data_leakage(train_set, val_set, test_set):
63
64     print("\n--- Checking for Data Leakage ---")
65
66     if not all([train_set, val_set, test_set]):
67         print("One or more sets are empty. Skipping leakage check.")
68         return True
69
70     train_paths = {item[0] for item in train_set}
71     val_paths = {item[0] for item in val_set}
72     test_paths = {item[0] for item in test_set}
73
74     leakage_train_val = train_paths.intersection(val_paths)
75     leakage_train_test = train_paths.intersection(test_paths)
76     leakage_val_test = val_paths.intersection(test_paths)
77
78     has_leakage = False
79     if leakage_train_val:
80         print(f"Error: Leakage detected between training and validation sets: {len(leakage_train_val)} samples.")
81         has_leakage = True
82     if leakage_train_test:
83         print(f"Error: Leakage detected between training and test sets: {len(leakage_train_test)} samples.")
84         has_leakage = True
85     if leakage_val_test:
86         print(f"Error: Leakage detected between validation and test sets: {len(leakage_val_test)} samples.")
87         has_leakage = True
88
89     if not has_leakage:
90         print("Success: No data leakage detected. The splits are mutually exclusive.")
91         return True
92
93     return False

```

▲圖片四：資料洩漏驗證的程式碼

二、執行結果截圖

```

● (.venv) wupeien@Mac hw1_dataset_split % python3 dataset_loader.py
Loading data from: /Users/wupeien/Documents/intro-ml-nycu-2025/data/MNIST

Dataset splitting complete.
Train set size: 41995
Validation set size: 8995
Test set size: 9010

--- Checking for Data Leakage ---
Success: No data leakage detected. The splits are mutually exclusive.

```

▲圖片五：終端執行結果

三、 問題與討論

問題一：為何最終驗證集與測試集的樣本數不是剛好均分 (如 8995 vs 9010)，而是有微小差異？

回答：差異來自與整數取捨與逐類分割的作用。程式在計算每個數字類別 0-9 的驗證集數量時，會使用 $\text{int}(\text{總數} * 0.15)$ ，這會無條件捨去小數點。而測試集則被分配了該類別中所有剩餘的樣本。當這個過程在 10 個類別上重複後，驗證集在每個類別中損失的小數部分，會被測試集累積起來，最終導致了兩者數量上的微小不平衡。

問題二：為何需要確保在進行正規化 (normalization) 或其他預處理之前分割資料集？

回答：為了避免資料洩漏 (Data Leakage)。如果先對整個資料集進行正規化，那麼計算正規化參數 (如平均值、標準差) 時就會使用到所有資料，包括訓練、驗證與測試集。這會導致測試集的資訊「洩漏」到訓練集中。模型在一個已經「偷看」過測試資料特徵的訓練集上學習，其評估結果會過於樂觀且不真實。正確的流程是先分割，然後僅用訓練集的數據計算正規化參數，再將此參數應用到所有三個集合上。